Base de Dados - Exame

Exame Normal - 24/25

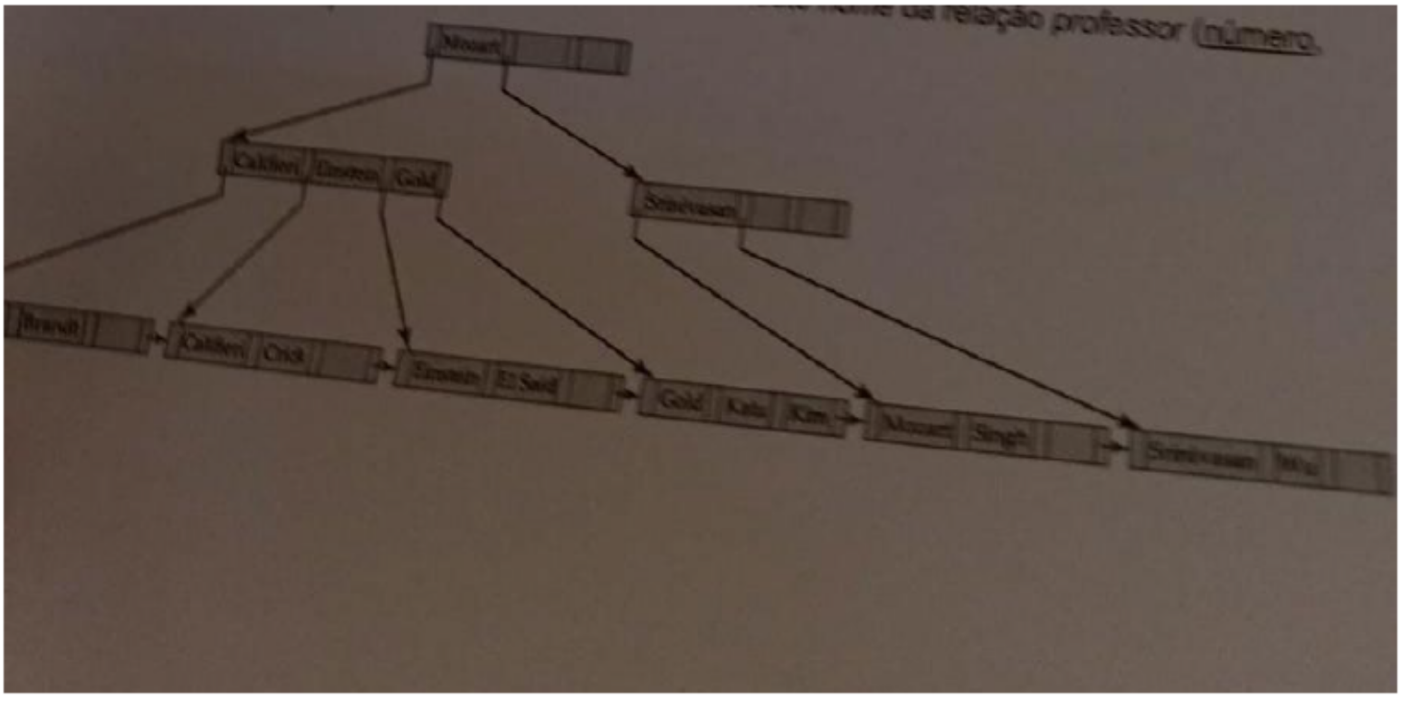
Grupo I (3 val)

Assinale com Verdadeiro (V) ou Falso (F). Resposta correta: +0,3. Resposta errada: -0,15, sem resposta: 0.

Item	Afirmação	V/F
1.1	Para passar da 2FN para a 3FN é necessário decompor relações. Removemos dependências o que diz a velocidade	V
1.2	Em álgebra relacional é possível combinar tuplos de diferentes relações. Com joins, unions e o produto cartesiano	V
1.3	Num Schedule conflito serializável é relevante a ordem das instruções read(Q) read(Q) entre duas transações. não há conflito porque ambos apenas leem	F
1.4	Uma relação não pode ter várias chaves candidatas. Pode ter multiplos, uma pessoa pode ter NIF e Email como candidatos	F
1.5	Os índices B+Tree são ordenados e, por isso, são adequados para testar desigualdades. Permitem buscas eficientes com <, >, ≤, ≥, alem de igualdades	V
1.6	No modelo relacional, os conceitos de esquema da relação e instância de relação têm significados diferentes. Esquema: estrutura/definição (atributos) Instância: dados concretos num momento específico.	V
1.7	Em SQL, todas as comparações com o valor null são avaliadas para true. resultam em NULL/UNKNOWN. Ex: 5 > NULL) -> NULL	F
1.8	Uma query SQL não é sempre executada da mesma forma pelo SGBD, dependendo do estado da BD (e.g., dados, estrutura, índices, etc.). O optimizador escolhe planos diferentes baseados em	V
1.9	Uma tabela não pode ter mais do que um clustering index. define a ordem física dos dados e só pode haver uma ordem	V
1.10	Um natural join é uma extensão do operador de join que evita a perda de tuplos que não tenham correspondência com a outra relação. O Natural join é um inner join (perde tuplos sem correspondencia) A descrição refere-se ao outer joins!	F

Grupo II (1 val)

Considere a seguinte B+Tree que representa um índice no atributo nome da relação professor (<u>número</u>, nome, departamento, salário).



Considere também a seguinte query SQL:

```
SELECT nome, departamento
FROM professor
WHERE nome = 'Einstein'
```

Explique de que forma o índice é percorrido para dar resposta à query. Indique todos os passos usados para percorrer a B+-Tree e indique, caso seja necessário para responder à query, a localização do ponteiro para aceder ao registo.

Para encontrar 'Einstein'

1) Nó raiz: 'Einstein' < 'Mozart' → seguir ponteiro esquerdo
2) Nó interno: Comparar com chaves deste nó
   - 'Crick' < 'Einstein' < 'Gold' → seguir ponteiro do meio
3) Nó folha: Encontrar 'Einstein' no folha
   - Localizar entrada para 'Einstein'
4) Aceder ao registo: Seguir o ponteiro do nó folha para aceder ao registo completo em disco
   - índice só tem atributo 'nome'
   - Para obter 'departamento', é necessário aceder ao registo através do ponteiro

Grupo III (1 val)

Explique os conceitos de superchave, chave candidata e chave primária. Ilustre a sua resposta apresentando um exemplo de uma relação e as respetivas chaves (superchave, candidata e primária).

Definições:

Superchave: conjunto de atributos que identifica univocamente cada tuplo. Pode conter atributos desnecessários

Chave Candidata: superchave mínima (não pode remover nenhum atributo sem perder unicidade).

Chave Primária: chave candidata escolhida pelo designer para identificar tuplos. Não pode ter valores NULL.

Exemplo:

Aluno (num_aluno, email, nome, telefone)

- Superchaves:
  - {num_aluno}
  - {email}
  - {num_aluno, nome} ← redundante
  - {num_aluno, email, nome, telefone} ← redundante
- Chave candidata:
  - {num_aluno}
  - {email}
- chave primária: {num_aluno} ← escolhido pelo designer (ficaria subentendido)

Grupo IV (1 val)

No contexto da gestão de transações, consideram-se como críticas as propriedades ACID. Explique o significado de cada uma destas propriedades, ilustrando a sua resposta com exemplos.

A - Atomicity (Atomicidade) transação é tudo ou nada. Se uma parte falha, tudo é revertido.

Exemplo: transferência bancária de 100€ da conta A para B:

- Débito de A: - 100 €
- Crédito de B: + 100 €
- Se o sistema falha após débito, a atomicidade garante rollback (ou 100 € voltam para A)

C - Consistency (Consistência) transação leva a BD de um estado válido para outro estado válido, respeitando todas as restrições.

Exemplo: Restrição: saldo ≥ 0

- tentativa de levantar 200 € com saldo de 50 € → transação rejeitada
- A BD mantém-se consistente

I - Isolation (Isolamento) transações concorrentes não interferem uma com as outras. O resultado é como se executassem sequencialmente.

Exemplo: t_1 e t_2 acedem a mesma conta simultâneamente.

- Sem isolamento: obtemos leituras sujas, lost updates
- Com isolamento: cada transação vê um estado consistente

D - Durability (Durabilidade) após commit, mudanças são permanentes mesmo com falhas no sistema

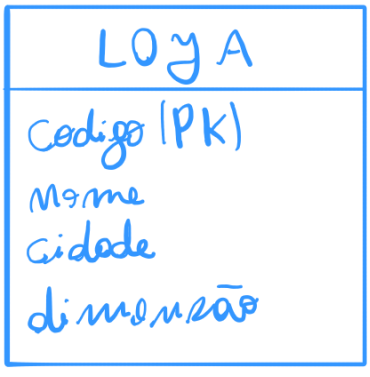
Exemplo: Após confirmar pagamento

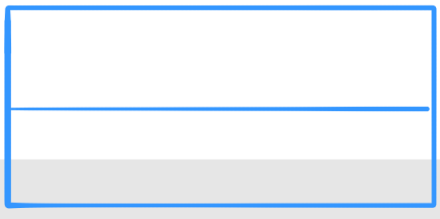
- Sistema falha 1 segundo depois
- Ao reiniciar, o pagamento continua registado (não se perde)

Grupo V (2 val)

A BikeX é uma empresa que aluga bicicletas e tem várias lojas espalhadas pelo país. Cada loja tem um código único, um nome, cidade e dimensão. Cada bicicleta é identificada por um número de série único, tem um modelo e a sua condição atual (e.g., 'boa', 'precisa de manutenção'). As bicicletas podem ser elétricas ou convencionais. As bicicletas elétricas podem ter mais do que um motor, sendo apenas necessário registar a potência de todos os motores em Watts. As bicicletas convencionais podem estar associadas a uma ou mais lojas, desde que sejam na mesma cidade, mas as bicicletas elétricas estão sempre associadas a uma loja.

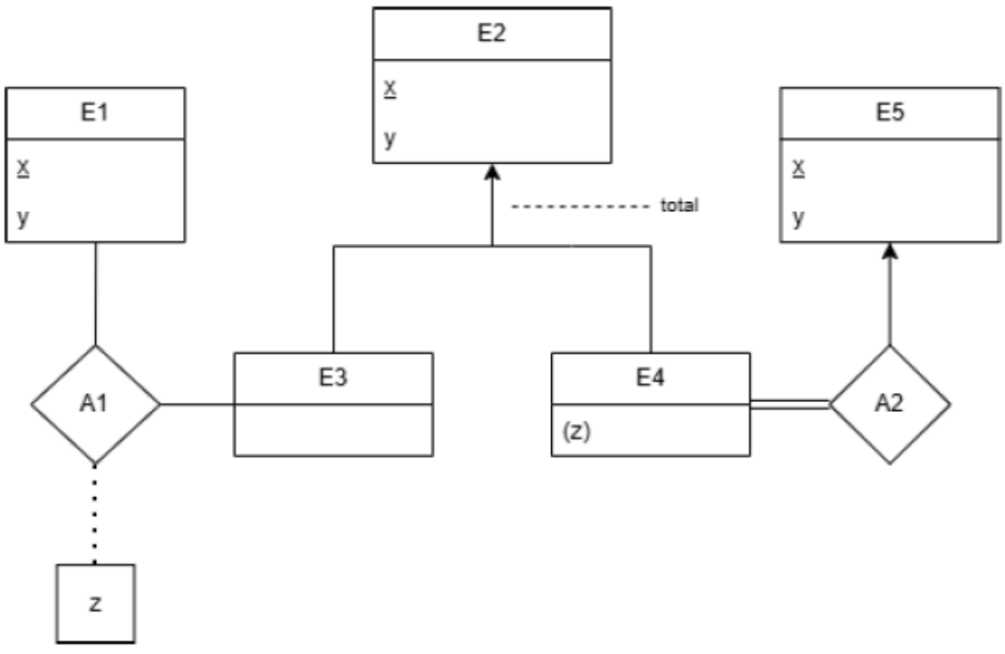
Elabore o modelo Entidade Associação (ER) que descreva o cenário apresentado. Assuma que uma bicicleta não pode ser ao mesmo tempo convencional e elétrica e que não existem mais tipos de bicicletas. Indique, caso necessário, todas as restrições de integridade.





Grupo VI (3 val)

Considere o seguinte modelo Entidade Associação (ER):



II.a) (1 val) Descreva, em português corrente, o cenário representado pelo modelo.

O modelo representa um sistema com 5 entidades (E1 a E5) relacionadas por duas associações (A1 e A2)

- E1

II.b) (2 val) Converta para o modelo relacional.

Grupo VII (2 val)

Considere o seguinte modelo relacional:

```
curso (nome_curso, area_cientifica, grau)
aluno (num_aluno, nome, idade, nome_curso)
        FK(nome_curso) -> curso
```

avg: média
min: mínimo
max: máximo
sum: soma
count: contagem

Para cada uma das alíneas seguintes, apresente a expressão de álgebra relacional:

VII.a) (0,5 val) Encontre os nomes de todos os alunos que têm mais de 20 anos.

$\Pi_{nome}(\sigma_{idade > 20}(alunos))$

VII.b) (0,5 val) Liste os nomes dos alunos e o nome do curso, para todos os alunos que estudam em cursos da área científica de "Economia".

$\Pi_{nome, nome_curso}(alunos \bowtie \sigma_{area_cientifica = "Economia"}(cursos))$

$\Pi_{aluno.nome, aluno.nome_curso}(\sigma_{aluno.nome_curso = curso.nome_curso \wedge area_cientifica = "Economia"}(alunos \times cursos))$

VII.c) (0,5 val) Liste os nomes e a idade de todos os alunos que são mais velhos do que todos os alunos do curso "LEI".

$\Pi_{nome, idade}(alunos) - \Pi_{a1.nome, a1.idade}(\sigma_{a1.idade \leq a2.idade}(\rho_{a1}(alunos) \times \sigma_{nome_curso = "LEI"}(\rho_{a2}(alunos))))$

VII.d) (0,5 val) Para cada curso, indique o número de alunos e a idade média.

$nome_curso\ \mathcal{G}\ count(num_aluno) \to num_alunos, avg(idade) \to idade_media(alunos)$

VII c) Alternativa

$idade_max_LEI \leftarrow \Pi_{idade}(\sigma_{nome_curso = "LEI"}(alunos))$

$\Pi_{nome, idade}(\sigma_{idade > ALL(idade_MAX_LEI)}(alunos))$

Grupo VIII (4 val)

Considere o seguinte modelo relacional:

```
curso (nome_curso, area_cientifica, grau)
aluno (num_aluno, nome, idade, nome_curso)
        FK(nome_curso) -> curso
```

Para cada uma das alíneas seguintes, apresente a respetiva **query SQL**:

VIII.a) (0,75 val) Encontre os nomes de todos os alunos que têm mais de 20 anos.

```
SELECT nome
FROM aluno
WHERE idade > 20;
```

VIII.b) (0,75 val) Liste os nomes dos alunos e o nome do curso, para todos os alunos que estudam em cursos da área científica de "Economia".

```
SELECT a.nome, a.nome_curso
FROM aluno a
JOIN curso C ON a.nome_curso = c.nome_curso
WHERE c.area_cientifico = "Economia";
```

VIII.c) (0,75 val) Liste os nomes e a idade de todos os alunos que são mais velhos do que todos os alunos do curso "LEI".

VIII.d) (1 val) Para cada curso, indique o número de alunos e a idade média.

VIII.e) (0,75 val) Encontre todas as áreas científicas que oferecem cursos em todos os graus disponíveis na área científica "Gestão".

VIII c)

```
SELECT nome, idade
FROM aluno a
WHERE idade > ALL(
        SELECT idade
        FROM aluno
        where nome_curso = 'LEI'
);
```

VIII d)

```
SELECT nome_curso, COUNT(*) AS num_alunos, AVG(idade) AS idade_media
FROM aluno
GRUPE By nome_curso;
```

VIII e)

```
SELECT area_cientifica
FROM curso
WHERE area_cientifica <> 'GESTÃO'
GROUP BY area_cientifica
HAVING COUNT(DISTINCT grau) = (
    SELECT COUNT(DISTINCT grau)
    FROM curso
    WHERE area_cientifica = 'GESTÃO'
);
```

OUTRA VERSÃO

```
SELECT DISTINCT c1.area_cientifica
FROM curso c1
WHERE NOT EXISTS
    SELECT *
    FROM curso
    WHERE area_cientifica = "GESTÃO"

    EXCEPT

    SELECT *
    FROM curso c2
    WHERE c2.area_cientifica = c1.area_cientifica
```